

QUY ĐỊNH VỀ XÂY DỰNG HỆ THỐNG BÀI TẬP

I. Nguyên tắc cốt lõi

1. Đóng HOW - Mở WHAT & WHY

- **Ý nghĩa:** Chấm dứt triệt để kỷ nguyên đào tạo "thợ gõ code" thụ động. Giảng viên là người ra đề bài, sinh viên là người thiết kế giải pháp.
- **Quy định bắt buộc:**
 - **Không bao giờ** viết các bước hướng dẫn giải thuật (Ví dụ: cấm dùng các câu như "Bước 1: Dùng vòng lặp for, Bước 2: Dùng if/else")
 - **Chỉ được phép** mô tả đích đến (What) và lý do khách hàng cần tính năng đó (Why). Sinh viên tự chọn công cụ lập trình để hoàn thành.

2. Tư duy đi trước, Code bước theo sau

- **Ý nghĩa:** Code chỉ là bản dịch của tư duy logic. Nếu logic sai, code vô giá trị.
- **Quy định bắt buộc:**
 - Mọi bài tập (từ Vận dụng đến Sáng tạo) đều phải yêu cầu sinh viên nộp Bản phân tích/Thiết kế luồng cùng source code
 - Đánh giá phân phân tích logic (sơ đồ, xác định input/output, test cases) tương đương hoặc cao hơn code.

3. Nhúng "Tư duy sản phẩm" vào mọi dòng code

- **Ý nghĩa:** mọi tính năng đều phải phục vụ một nghiệp vụ thực tế
- **Quy định bắt buộc:**
 - Xóa sổ các bài tập toán học vô tri (Ví dụ: Tìm UCLN, in mảng số nguyên).
 - **100% đề bài** phải có bối cảnh dự án, đóng vai nhân vật (Roleplay) và giải quyết một "nỗi đau" cụ thể của người dùng (Ví dụ: Xử lý giỏ hàng bị lỗi, lọc tin nhắn rác, tối ưu truy vấn cơ sở dữ liệu).

4. Luôn cài cắm "Bẫy dữ liệu"

- **Ý nghĩa:** Rèn luyện tính cẩn thận và tư duy lường trước rủi ro của một kỹ sư phần mềm thực thụ.
- **Quy định bắt buộc:**

- Trong yêu cầu nghiệm thu, giảng viên phải đưa ra ít nhất 1-2 kịch bản dữ liệu dị biệt (Ví dụ: Người dùng nhập khoảng trắng, nhập số âm, hoặc kết nối mạng bị đứt giữa chừng).
- Đòi hỏi sinh viên phải có cơ chế xử lý lỗi và đưa ra thông báo thân thiện cho người dùng, thay vì để hệ thống "crash".

5. Nâng cấp dần "Quyền tự quyết" theo 3 cấp độ

- **Ý nghĩa:** Đảm bảo sinh viên không bị ngợp nhưng cũng không được phép lười biếng.
- **Quy định bắt buộc:**
 - **Vận dụng cơ bản:** Thiết kế theo phương pháp **Sửa lỗi nghiệp vụ** hoặc **Điền khuyết**. Thay vì bắt sinh viên xây nhà từ bãi đất trống, ta giao cho các bạn một ngôi nhà đã xây thô nhưng bị... thùng mái, và yêu cầu các bạn đọc bản vẽ để vá lại.
 - **Vận dụng chuyên sâu:** Sinh viên phân tích được yêu cầu, ra được 1 giải pháp, thiết kế các bước thực hiện và triển khai code.
 - **Phân tích:** Sinh viên phân tích được yêu cầu, đưa ra được tối thiểu 2 giải pháp có thể thực hiện, so sánh (Ưu điểm, nhược điểm) 2 giải pháp, lựa chọn 1 giải pháp phù hợp để thiết kế các bước thực hiện, triển khai code
 - **Sáng tạo:** Sinh viên tự quyết định kiến trúc toàn hệ thống và tự định nghĩa bài toán từ một yêu cầu mở

6. Hệ sinh thái nghiệp vụ đồng nhất

- **Ý nghĩa:** Tránh hội chứng "chuyển đổi ngữ cảnh" (Context-switching) gây quá tải nhận thức cho sinh viên. Giúp sinh viên không chỉ học code mà còn "thấm" sâu kiến thức nghiệp vụ của một lĩnh vực cụ thể (ví dụ: E-commerce, EdTech, FinTech, Logistics...), từ đó hiểu rõ vòng đời của một phần mềm thực tế
- **Quy định bắt buộc:**
 - Toàn bộ chuỗi bài tập trong một session hoặc các session liên kế nhau phải được đặt trong cùng một lĩnh vực
 - Dữ liệu và kết quả của bài tập cấp độ thấp phải là tiền đề, hoặc là một phần module của bài tập cấp độ cao hơn.

STT	Cấp độ bài tập	Số lượng	SV mục tiêu	Ước lượng thời gian
1	Vận dụng cơ bản (Đọc hiểu, sửa lỗi/Điền khuyết code)	2 bài	SV Yếu (Mục tiêu qua môn – Điểm 5-6)	120 – 150 phút
2	Vận dụng chuyên sâu (Tự thiết kế luồng & code từ đầu)	1 bài	SV Trung bình (Mục tiêu điểm 7)	150 – 180 phút
3	Phân tích (Đa giải pháp, lập bảng so sánh)	1 bài	SV Khá (Mục tiêu điểm 8)	180 phút
4	Sáng tạo (Thiết kế giải quyết bài toán mở)	1 bài	SV Giỏi/Xuất sắc (Mục tiêu điểm 9.10)	180 phút

III. Tiêu chuẩn ra bài tập

1. Vận dụng cơ bản

- **Mục tiêu:** Rèn luyện kỹ năng đọc hiểu mã nguồn (legacy code), khả năng dò luồng (trace code) và phát hiện lỗi hỏng logic nghiệp vụ hoặc bẫy dữ liệu.
- **Ý nghĩa tổng quát:** Đây là bước đệm tâm lý giúp sinh viên yếu vượt qua "hội chứng trang giấy trắng". Thay vì tự xây nhà, các bạn học cách đối chiếu giữa "yêu cầu của khách hàng" và "đoạn code đang có" để vá lỗ hỏng, từ đó nhận thức rõ hậu quả của việc thiếu tư duy chặn lỗi (Edge Cases).
- **Cấu trúc bài tập:**
 - **Bối cảnh nghiệp vụ:** Nêu rõ quy tắc đúng của hệ thống.
 - **Vấn đề hiện tại:** Khách hàng đang phàn nàn điều gì về tính năng này?
 - **Hiện trường giả:** Cung cấp một đoạn source code chạy được (không lỗi cú pháp) nhưng sai logic hoặc thiếu điều kiện ràng buộc.
 - **Yêu cầu đầu ra:** Sinh viên phải nộp 2 phần: (1) Chỉ ra đoạn code sai bằng dữ liệu test case cụ thể, và (2) Source code đã được sửa chuẩn.

2. Vận dụng chuyên sâu

- **Mục tiêu:** Sinh viên có khả năng dịch thuật chính xác từ ngôn ngữ nghiệp vụ (đời thực) sang ngôn ngữ logic (máy tính) và tự thiết kế các bước thực hiện tuần tự.
- **Ý nghĩa tổng quát:** Định hình kỷ luật "Tư duy đi trước, Code bước theo sau". Sinh viên rèn luyện kỹ năng tìm hiểu và phân tích – phân rã yêu cầu bài toán để đưa ra được đầu vào/đầu ra rõ ràng, đưa ra được giải pháp để giải quyết bài toán từ đó thiết kế các bước thực hiện và triển khai code

▪ **Cấu trúc bài tập**

- **Bối cảnh & Vấn đề:** Nêu rõ câu chuyện nghiệp vụ. Khách hàng đang cần module/tính năng gì và tại sao lại cần nó?
- **Yêu cầu nghiệp vụ:** Các quy tắc hệ thống cần tuân thủ (Ví dụ: Công thức tính toán, các trường hợp tính phí/giảm giá)
- **Ràng buộc & Bẫy dữ liệu:** Đặt ra 1-2 kịch bản dữ liệu dị biệt để thử thách tư duy lường trước rủi ro (Ví dụ: "Hệ thống sẽ phản hồi ra sao nếu tài khoản khách hàng chưa được kích hoạt?").
- **Yêu cầu đầu ra:**
 - ❖ Báo cáo phân tích và thiết kế giải pháp
 - ✓ Phân tích bài toán (I/O): Sinh viên phải định nghĩa rõ Dữ liệu đầu vào (Input) cần những gì, kiểu dữ liệu nào và Kết quả mong đợi (Output) hiển thị ra sao.
 - ✓ Đề xuất giải pháp: Trình bày ngắn gọn ý tưởng logic để giải quyết nghiệp vụ trên (Ví dụ: "Sẽ dùng cấu trúc rẽ nhánh để kiểm tra hạng thành viên, sau đó dùng vòng lặp để tính tổng giỏ hàng").
 - ✓ Thiết kế các bước : Vẽ lưu đồ hoặc gạch đầu dòng tuần tự các bước xử lý
 - ❖ Triển khai và chống lỗi
 - ✓ Source code hoàn chỉnh. Logic code bắt buộc phải "khớp" với các bước đã thiết kế
 - ✓ Có các đoạn code cụ thể để chặn và xử lý "Bẫy dữ liệu" mà giảng viên đã nêu.

3. Phân tích

- **Mục tiêu:** Sinh viên có khả năng nhìn nhận một bài toán dưới nhiều góc độ kỹ thuật khác nhau, biết cách đánh giá sự đánh đổi (Trade-off) để đưa ra quyết định tối ưu.
- **Ý nghĩa tổng quát:** Rèn luyện tư duy phản biện. Sinh viên hiểu rằng trong lập trình có nhiều con đường để đi đến cùng một kết quả, nhưng người kỹ sư giỏi là người biết phân tích ưu/nhược điểm của từng con đường và chọn ra lối đi phù hợp nhất với bối cảnh thực tế.
- **Cấu trúc bài tập**
 - **Bối cảnh & Yêu cầu bài toán:** Đưa ra một yêu cầu nghiệp vụ bình thường nằm trong Domain Knowledge. (Ví dụ: Viết tính năng lọc danh sách sản phẩm theo giá).
 - **Quy tắc nghiệp vụ:** Các điều kiện tính toán, phân loại hệ thống cần tuân thủ.

- **Ràng buộc & Bẫy dữ liệu:** Cài cắm các kịch bản ngoại lệ để sinh viên phải xử lý.
- **Yêu cầu đầu ra:**
 - ❖ Phân tích & Đề xuất (Đa giải pháp)
 - ✓ Phân tích yêu cầu để xác định rõ Đầu vào (Input) / Đầu ra (Output).
 - ✓ Đưa ra **tối thiểu 2 giải pháp** có thể giải quyết được bài toán (Ví dụ: Giải pháp 1 dùng hai vòng lặp lồng nhau; Giải pháp 2 dùng cấu trúc dữ liệu Map/Dictionary).
 - ❖ So sánh & Lựa chọn
 - ✓ Lập bảng so sánh rõ ràng: **Ưu điểm - Nhược điểm** của từng giải pháp (về tốc độ chạy, bộ nhớ, độ dễ hiểu, dễ bảo trì...).
 - ✓ Chốt lựa chọn 1 giải pháp phù hợp nhất với bối cảnh đề bài.
 - ❖ Thiết kế & Triển khai
 - ✓ Thiết kế các bước thực hiện (luồng logic/mã giả) cho giải pháp đã chọn.
 - ✓ Triển khai viết source code hoàn chỉnh, đảm bảo bắt mọi bẫy dữ liệu và ngoại lệ.

4. Sáng tạo

- **Mục tiêu:** Sinh viên làm chủ vòng đời phát triển của một tính năng hoặc sản phẩm nhỏ: tự định nghĩa yêu cầu, thiết kế kiến trúc và hoàn thiện sản phẩm.
- **Ý nghĩa tổng quát:** Đây là bài kiểm tra toàn diện về "Tư duy sản phẩm". Sinh viên không giải quyết một "đề bài", mà giải quyết một "nỗi đau" mở của khách hàng, tích hợp toàn bộ các module từ các bài tập trước đó thành một cỗ máy hoạt động trơn tru.
- **Cấu trúc bài tập:**
 - **Vấn đề:** Đưa ra một nhu cầu lớn của khách hàng chưa được định hình chi tiết.
 - **Ràng buộc kỹ thuật:** Các giới hạn về công nghệ, thời gian thực thi, hoặc yêu cầu về tổ chức code (Clean Code, tách hàm/class).
 - **Yêu cầu đầu ra:**
 - ❖ Thiết kế Kiến trúc: Xác định các module cần có và luồng dữ liệu (Data flow).
 - ❖ Sản phẩm hoàn chỉnh: Source code xử lý mượt mà mọi ngoại lệ, không bị crash, giao tiếp thân thiện với người dùng.